Assignment 7: Observe, Instrument, Diagnose, and Explain Process Memory

Linux Systems and Performance - Week 7

Address-Space Dump Tool -> malloc Instrumentation -> Memory Anomaly Diagnosis

1. Assignment Philosophy

Build a process-memory analyzer and a malloc tracing library. Use them to measure memory behavior, diagnose three case programs, and explain your conclusions with evidence.

BUILD -> OBSERVE -> HYPOTHESIZE -> VERIFY -> EXPLAIN

Important: AI coding tools may be used. However, you are responsible for understanding, debugging, modifying, and explaining all submitted code and measurement results. Every number in your report must come from your own runs on your own machine.

Recommended: **Include screenshots of programs you ran yourself in your report, showing the commands and key results.**

Note: Several questions ask why Size differs from Rss, or why allocated memory does not appear as resident. The full mechanism (page faults and demand paging) is the topic of Week 8. For this assignment, report what you observe and state a hypothesis; a complete explanation is not expected yet.

2. Challenge Levels

Level	Build	Run / Trigger	Measure / Observe	Main Question
Level 1	vasdump: /proc maps and smaps analyzer	sleep, your own C program, python3	per-category Size / Rss / Pss; ELF segment to VMA mapping	What backs each region of a process's address space?
Level 2	libmtrace_lite.so: LD_PRELOAD malloc wrapper	ls, python3, multithreaded program, edge-case calls	call counts, size histogram, peak and live bytes, leak candidates	What does a program ask of malloc, and how does that appear in the VMAs?
Level 3	diagnosis reports for instructor-provided case programs	symptom-only programs (case1 to case5)	time series, strace, smaps, your own tools; before / after	How do you tell apart different causes of the same symptom?

Complete all three levels. Use vasdump from Level 1 and libmtrace_lite.so from Level 2 to investigate the Level 3 cases.

3. Common Interface and Rules

- Record kernel, glibc, architecture, page size, compiler, build flags, and allocator settings in README. Use uname -r, ldd --version, uname -m, and getconf PAGESIZE.
- Implement `vasdump` in any language available on the course machines. Implement `libmtrace_lite.so` in C.
- Do not use existing tools as the implementation. `pmap`, `smem`, process-inspection libraries, Valgrind, `mtrace`, and `ltrace` may not replace Level 1 or Level 2 code. You may use them to cross-check your own output.
- `Wrap the original glibc allocator; do not replace it. Handle allocations made before dlsym completes using the bootstrap support described in Section 6.2.`
- Use kB = 1024 bytes for memory comparisons. Convert library byte counts by dividing by 1024 and label every column.
- Repeat each measurement at least three times when the value can vary between runs (for example under ASLR or with threads), and report the median or the range.
- Compare fresh processes under the same build, workload, and settings. Save each run separately. Measure timings without strace; collect traces in separate runs.
- Record exact commands, environment variables, and configuration values. Raw dumps or screenshots without interpretation are not sufficient.

4. Required Tool Interfaces and Provided Materials

4.1 vasdump Interface

```
./vasdump [options] <pid>
./vasdump [options] -- <command> [args...]
```

PID mode: inspect the process without terminating it.

Command mode: start the command, wait --delay seconds, take the snapshot(s), then terminate and reap the child. Use SIGTERM and a bounded SIGKILL fallback.

On invalid arguments, exec/read failures, or early target exit, print a diagnostic and return nonzero. Clearly mark incomplete snapshots.

- Diff mode: take the first snapshot after --delay in command mode, or immediately in PID mode. Take the second snapshot after the specified interval.
- Match VMAs by address range. Show added/removed ranges, signed changes in Size/Rss, and changed category totals. A split or merge may appear as removals and additions.
- Sort by absolute change in Rss, then Size, then starting address. Use zero for a range missing from one snapshot.

Option	Purpose	Notes
-v	print every VMA with its fields	one line per VMA: address range, permissions, offset, pathname, Size, Rss, Pss
--delay <sec>	wait time before dumping in `--` mode	default 1 second
--diff <sec>	take two snapshots <sec> seconds apart and report the changes	two snapshots; added/removed/changed VMAs and category deltas

- Example output (values are illustrative):

```
./vasdump 12345
category  Size_kB  Rss_kB  Pss_kB  ...
[heap]       2048    1100    1100  ...
TOTAL         ...     ...     ...  ...
# Remaining columns: Shared_Clean, Shared_Dirty,
# Private_Clean, Private_Dirty, Anonymous (kB).
./vasdump --diff 5 12345
status  range              delta_Size_kB delta_Rss_kB
added   70000000-71000000          +16384       +16384
...
category delta totals: ...
```

4.2 libmtrace_lite Interface

```
MTRACE_OUT=trace.log LD_PRELOAD=./libmtrace_lite.so <command> [args...]
```

- MTRACE_OUT selects the output file; the default is stderr. If opening the file fails, report the error and use stderr. Preserve program stdout.
- Test single dynamically linked processes that do not already use SIGUSR1. Print a summary on normal exit and support snapshots before termination; abnormal termination may skip the exit summary.

Setting	Purpose	Notes
MTRACE_OUT	file that receives the summary	default: stderr
SIGUSR1	request a prompt snapshot of current counters	handler notifies; reporting occurs safely outside the handler (Section 6.2)
MTRACE_INTERVAL	print a snapshot every n seconds	document how you avoided re-entering malloc

```
MTRACE_OUT=trace.log LD_PRELOAD=./libmtrace_lite.so ./checkpoint_workload
PID=12345 READY checkpoint=baseline
# In another terminal: kill -USR1 12345; wait for snapshot, then ./vasdump 12345
# Press Enter in the workload terminal only after both observations complete.
Example allocator snapshot:
snapshot=3 pid=12345 accounting=requested units=bytes
calls malloc=1000 calloc=0 realloc=0 free=0
live_blocks=1000 live_bytes=1024000 peak_live_bytes=1024000
histogram: 0=0 1=0 2-3=0 ... 1024-2047=1000 ...
```

4.3 Provided Materials

- Provided files: case1-case3 binaries, optional case4/case5 binaries, an ELF/checkpoint workload, known-count and edge-case tests, and a Level 2 bootstrap/signal support skeleton.
- Case sources are withheld until the deadline. Implement your own VMA analyzer and allocation statistics. Identify supplied code and your changes in README.
- Run ./caseN --help for options and ./caseN for the default workload. Wait for PID and READY before sampling; use phase markers to align measurements.
- Follow the package README for run duration, memory limits, termination, and supported environments. Use the supplied binary for your architecture or the supported course environment.

4.4 Practical Notes

- Use `readelf -lW <executable>` for program headers and `cat /proc/<pid>/maps` to compare them with the runtime mappings.
- A process that has exited has no `/proc` entry. Keep test programs alive (for example with `sleep()` or by waiting on stdin) so that you can inspect them.
- Reading another user's `/proc/<pid>/smaps` may be denied. Use your own processes.
- ASLR randomizes addresses. To compare two runs with identical layout you may disable it for one command with `setarch "$(uname -m)" -R <command>`, but also report the default behavior.

5. Level 1 - Build an Address-Space Dump Tool

5.1 Required Architecture

```
/proc/<pid>/maps  ----\
/proc/<pid>/smaps -----> parse VMAs -> classify -> aggregate -> report
readelf -lW <exe> ----/   (PT_LOAD segments, matched against the VMAs in your report)
```

5.2 Required Implementation

- Parse every VMA header line (address range, permissions, offset, device, inode, pathname) and the per-VMA fields that follow it in smaps.
- Classify each VMA once: main executable, [heap], main [stack], shared library, other file-backed mapping, anonymous mapping, or special/other bracketed mapping.
- Identify the executable using /proc/<pid>/exe and device/inode where available. Match a trailing (deleted) suffix without removing it from displayed paths.
- Apply named heap/stack/special rules first, then file rules. Unnamed mappings are anonymous; unmatched bracketed names are special/other.
- Keep an anonymous .bss tail in the anonymous category and identify its ELF relationship in Section 5.4.
- For each category, each permission string, and the overall process, report Size, Rss, Pss, Shared_Clean, Shared_Dirty, Private_Clean, Private_Dirty, and Anonymous.
- Include all observed permission strings. Category totals and permission totals must each equal the overall totals. The Anonymous field is not the total size of anonymous VMAs.
- Handle pathnames that contain spaces, the `(deleted)` suffix, and a process that exits while you are reading it. Do not assume a fixed number or order of VMAs.
- Implement the options listed in Section 4.1.

5.3 Required Functional Experiments

- Print a stage marker and pause at each measurement point. Record allocation sizes and whether pages were touched.
- Allocation experiments: touch at least one byte per page before measuring. Recursion: touch a local array in each frame, prevent tail-call elimination, and record depths and stack limits.
- Thread experiment: keep four workers alive and state whether they allocate memory.

Demonstrate all of the following and attach the `vasdump` output for each:

Experiment	Required Trigger	What to Explain
Idle process	`./vasdump -- sleep 1000`, and your own minimal C program that only sleeps	why even a trivial process has many VMAs, and which belong to the program and which to the loader and libraries
Small vs. large allocation	allocate 1,000 blocks of 1 KiB, then one 16 MiB block; dump after each step	where each allocation appears ([heap] or an anonymous mapping) and why
Threads	start 4 threads that stay alive, then dump	the extra anonymous VMAs per thread and the `---p` regions next to them
Stack depth	recurse to two different depths and dump at the deepest point of each	how [stack] changes with recursion depth
Growth over time	a program that allocates and touches memory every second; run `--diff 5`	which categories changed, by how much, and why

5.4 ELF Segment to VMA Mapping

Build the provided ELF workload or an equivalent program with -fPIE -pie. Include an initialized global, a read-only object, and an uninitialized 8 MiB array. Print their addresses, including both ends of the array, and keep the process alive.

Save readelf -hW, readelf -lW, readelf -SW, and nm -n output.

Calculate load bias = runtime object address - ELF symbol value. Verify it with a second symbol. Calculate runtime segment addresses as load bias + p_vaddr.

Check one VMA against the page-aligned PT_LOAD address and file offset. The first VMA start equals load bias only if its aligned p_vaddr is zero.

Complete the table for your actual PT_LOAD entries. Explain page alignment, permission splits such as RELRO, and any anonymous .bss tail. PIE/non-PIE comparison is optional.

Segment	Offset / VirtAddr / FileSiz / MemSiz	Flags	Matching VMA(s) at Runtime	Notes
LOAD 1				
LOAD 2				
LOAD 3				
LOAD 4				

5.5 Required Explanation

1. Why is `libc.so` mapped as several VMAs with different permissions and offsets?
2. Which regions in your dump have a Size much larger than their Rss? State a hypothesis for why (Week 8 gives the full mechanism).
3. What is the difference between Rss and Pss, and which regions show a large gap between them? Why?
4. Where is `.bss` in your ELF table and in the VMA list? What does `MemSiz > FileSiz` imply about how it is mapped?
5. Run the same program twice. Which addresses change, which offsets stay the same, and what mechanism causes this?

6. Level 2 - Instrument malloc with an LD_PRELOAD Library

6.1 Required Architecture

```
application -> malloc / free / calloc / realloc
      |
      v
libmtrace_lite.so  (LD_PRELOAD wrapper: counters, size histogram, live-byte accounting)
      |
      v
dlsym(RTLD_NEXT, ...) -> original glibc malloc / free / calloc / realloc

summary printed at exit;  snapshot printed on SIGUSR1
```

6.2 Required Implementation

- Wrap malloc, free, calloc, and realloc. Resolve originals with dlsym(RTLD_NEXT, ...) and forward calls after initialization.
- **Recursion guard: use a thread-local guard and allocation-free metadata/output paths. Use write(2) with a stack buffer; preserve errno in free().**
- **Bootstrap: use the supplied skeleton or an equivalent design. Check size overflow, alignment, capacity, and concurrent initialization.**
- **Recognize bootstrap pointers in free/realloc; never pass them to glibc free or malloc_usable_size. Document their copy/lifetime rules and exclude bootstrap and observer allocations from statistics.**
- **Thread safety:** counters and accounting must stay consistent when several threads call the wrappers at the same time (atomics or a lock). Document your choice.
- **Accounting: track live blocks, live bytes, and peak live bytes. Choose requested-size accounting with a pointer table, or usable-size accounting with malloc_usable_size. Label the choice in each summary.**
- **Track ownership, read the old size before free/realloc invalidates it, and flag metadata exhaustion. Do not subtract untracked blocks or silently drop accounting entries.**
- **Edge cases: test free(NULL), malloc(0), zero-sized calloc, realloc(NULL,n), glibc realloc(p,0), calloc overflow, and nonzero realloc failure.**
- **Count each outer API call once, including failures. realloc(NULL,n) counts as realloc. Failed calls do not add live allocations; failed nonzero realloc preserves the old allocation. Track non-NULL zero-sized allocations until released.**
- **Summary at exit** (library destructor): the number of calls to each function; the request-size histogram in power-of-two buckets; peak live bytes; live blocks and live bytes at exit (leak candidates).
- Histogram: count successful malloc/calloc/realloc requests by requested bytes. Use the checked product for calloc and the new size for realloc. Exclude free, failed requests, and overflow.
- Use buckets 0, 1, 2-3, 4-7, 8-15, ... with a documented final bucket. Count each successful realloc once, including in-place changes. Remaining live allocations are leak candidates, not proof of a leak.
- **SIGUSR1: use the supplied notification support or a documented equivalent. The handler must preserve errno and must not lock, allocate, walk metadata, or use stdio.**
- **Print a consistent snapshot outside the handler, including while the workload is paused. Document reporting overhead and coalesced requests. A handler-based alternative must justify signal safety and state any consistency limits.**

6.3 Required Functional Experiments

Experiment	Required Trigger	What to Explain
ls	`LD_PRELOAD=./libmtrace_lite.so ls -l /usr`	the program output is unchanged; interpret the call counts and the size histogram
Python	`LD_PRELOAD=./libmtrace_lite.so python3 -c "print(sum(range(10**6)))"`	why the number of small allocations seen by malloc may be far below the number of objects Python creates
Multithreaded	your own program with at least 4 threads, each doing at least 100,000 malloc/free pairs	counts match the known number of calls (plus a small, explained number of runtime-internal calls); no crash or deadlock
Edge cases	a test program that makes the calls listed in Section 6.2	no crash; the statistics remain sensible

6.4 Level 1 + Level 2 Connection Experiment

Use the provided checkpoint workload or an equivalent. Record baseline before allocations, then checkpoints A-D in the same process. Touch each allocated page before A/B.

At each checkpoint: wait for the PID/stage marker, request a library snapshot, wait for its output, then run vasdump. Advance the workload only after both measurements finish.

Report totals and changes from baseline in kB. Include runtime/observer overhead and retain decimal precision when converting live bytes.

```
kill -USR1 <pid>
./vasdump <pid>
```

Checkpoint	live bytes (mtrace-lite)	[heap] Size	[heap] Rss	anonymous mappings (total Size)	Interpretation
A. after 1,000 x 1 KiB allocations					
B. after one more 16 MiB allocation					
C. after freeing the 16 MiB block					
D. after freeing all 1 KiB blocks					

Explain why the columns differ from each other and from checkpoint to checkpoint. State your hypotheses and what measurement would confirm them.

6.5 Required Explanation

6. Why can `printf` or `fprintf` inside a malloc wrapper recurse or deadlock, and how does your design avoid it?
7. How do you obtain the original function pointers, and what goes wrong if an allocation arrives before they are resolved?
8. What does your thread-safety choice cost on every allocation, and how accurate is "peak live bytes" when several threads allocate at the same time?
9. Name two kinds of memory use that `LD_PRELOAD` interposition cannot see, and show one of them in your `vasdump` output.
10. In your connection table, why does live bytes differ from [heap] Rss even though nothing leaked?

7. Level 3 - Diagnose Memory Anomalies

7.1 Required Method

```
symptom -> measure -> hypothesis -> rule out alternatives -> remedy -> re-measure
```

Support each diagnosis with evidence and at least one competing explanation. Test a remedy using the same workload. If a source change is necessary, use the prediction exception in Section 7.3. State any remaining uncertainty.

7.2 Provided Case Programs

Program	Reported Symptom	Status
case1	memory use keeps growing the longer the program runs	required
case2	many objects were freed, but Rss does not go down	required
case3	adding threads makes VSZ very large, but Rss stays small	required
case4	repeated allocation and release of large buffers costs a lot of CPU time	additional (bonus)
case5	a large allocation barely changes Rss until part of it is used	additional (bonus)

7.3 Required Evidence for Each Case

Use about one page per case. Each case section must contain all five items:

Item	What to Provide
Evidence	the exact commands and the key output lines; for symptoms that develop over time, a time series of at least 10 samples
Hypothesis	how the evidence leads to the cause
Alternatives ruled out	at least one competing explanation (for example real leak, allocator retention, or fragmentation) and the measurement that excludes it
Remedy and re-measurement	a remedy applied without modifying the binary (command-line option, environment variable, glibc tunable, or an LD_PRELOAD shim you write), with before/after numbers; if only a code change can fix it, describe the change precisely and predict its measurable effect
Confidence	how sure you are, and which additional measurement would settle the remaining doubt

7.4 Diagnosis Summary Table

Put this table at the start of your Level 3 section:

Case	Root Cause (one line)	Key Evidence	Remedy	Before -> After
case1				
case2				
case3				
case4				
case5				

7.5 Allowed Tools and Measurement Practice

- `strace -f -e trace=brk,mmap,munmap,mremap,madvise` shows how the program asks the kernel for memory.
- `pmap -x`, `/proc/<pid>/status` (VmSize, VmRSS), `smaps`, `smaps_rollup`, and your own `vasdump` and `libmtrace_lite` show the resulting state.
- `ltrace`, `gdb`, `readelf`, and (where available) `perf stat -e page-faults` can support your hypothesis.
- glibc malloc tunables (`GLIBC_TUNABLES=glibc.malloc.*`, `MALLOC_ARENA_MAX`), `malloc_trim`, and `malloc_stats` are useful for remedies and cross-checks.
- Sample at fixed intervals. For changing symptoms, collect at least 10 samples from READY.
- Save CSV with run ID, sample index, elapsed time from READY, phase, VmSize, and VmRSS (kB). Include allocator snapshot identifiers when used.
- Use fresh processes and matching phases for before/after comparisons. A simple memory sampling loop is shown below; add timestamps and phase information to your saved data.

```
while true; do grep -E 'VmSize|VmRSS' /proc/<pid>/status; sleep 5; done
```

7.6 Required Analysis

11. How can you tell a true leak from memory that was freed but is still held by the allocator? Which two numbers do you compare?
12. Why can Rss stay high after `free()`, and which conditions let glibc return memory to the kernel (for example trim threshold, mmap threshold, position in the heap)?
13. Why can VSZ be very large while Rss stays small? Which mappings account for the difference in your case3 dump?
14. Which of your conclusions would have been wrong or unprovable if you had measured only once, only Rss, or only the final state?

8. Evidence Required

Level	Implementation Evidence	Runtime / Measurement Evidence	Analysis Evidence
1	vasdump source + README	outputs for the five experiments; ELF segment to VMA table	VMA classification, Size / Rss / Pss, ELF mapping, ASLR
2	library source + Makefile + support attribution	ls / python3 / multithreaded / edge-case logs; connection table	recursion and bootstrap, thread safety, what LD_PRELOAD cannot see
3	diagnosis report for the three required cases	time series, strace and smaps evidence, before/after tables	leak vs. retention, VSZ vs. Rss, remedy validation

9. Submission Package

- Source code for `vasdump` and `libmtrace_lite.so`, with a Makefile for Level 2. Clearly identify the files that belong to each level.
- README: build/run commands, environment, output columns and units, supplied code and modifications, and a map from experiments to evidence files.
- Include source and build instructions for every student-written workload, test, or remedy shim.
- Report: 6-8 pages recommended, excluding logs. Organize by level and answer Sections 5.5, 6.5, and 7.6 using identifiers such as 5.5-Q1. Show selected annotated output and tables; submit complete logs and CSV separately.
- Level 1: `vasdump` output for each experiment in Section 5.3 and the completed ELF segment to VMA table.
- Level 2: logs of the four experiments in Section 6.3 and the completed connection table in Section 6.4.
- Level 3: the diagnosis summary table and one section for each required case (and for any optional case you completed).
- A short AI-use statement if AI tools were used, including at least one generated suggestion or code issue that you verified, changed, or rejected.

Suggested directory layout:

```
assignment7_<name>/
  level1/   vasdump source, README.md, samples/
  level2/   mtrace_lite.c, Makefile, logs/
  level3/   raw logs and time series for each case
  report.pdf   (or report.md)
```

10. Evaluation

Category	Weight	What Is Evaluated
Implementation	35%	correct and robust vasdump and libmtrace_lite: parsing, classification, recursion guard, thread safety, edge cases
Experiment & Measurement	30%	exact commands, repeated measurements, time series, before/after comparisons, the Level 1 + Level 2 connection table
Technical Explanation	25%	correct reasoning about VMAs, Rss and Pss, allocator behavior, the limits of LD_PRELOAD, and diagnostic logic
Code Quality & Report	10%	structure, reproducibility, concise evidence, honest AI-use statement

Grades depend on correct implementation, controlled measurements, and supported explanations. Different memory sizes, allocation paths, or small remedy effects are valid when explained. Optional work adds up to 10 bonus points. Copied measurements and unsupported conclusions receive no credit for the affected item.

11. Final Check-Off Questions

- Run `vasdump` on a process of your choice and explain its largest VMA by Size and by Rss.
- Point to one PT_LOAD segment in the `readelf` output and show the matching VMA(s) in your dump, including where `.bss` ends up.
- Before running the connection experiment, predict which of live bytes, [heap] Rss, and anonymous mappings will change after the 16 MiB allocation and after it is freed.
- Explain how removing the recursion guard or bootstrap support could fail. An optional isolated demonstration must use a separate test; submit the safe implementation.
- Name something your library cannot see and show it in the `vasdump` output.
- For one case, predict what a second measurement taken a minute later will show before you run it.
- Point to one `strace` or smaps line that supports or contradicts your hypothesis for one case.